

Ingénieurs 2^{ème} année TRAVAUX PRATIQUES MISE EN ŒUVRE D'UN DSP

THEMES ABORDES EN TP :

- PRISE EN MAIN DE LA CHAÎNE DE DEVELOPPEMENT
- GENERATION ET SYNTHESE DE SIGNAUX
- IMPLANTATION DE FILTRES NUMERIQUES RII puis RIF
- APPLICATIONS DIVERSES (CAG, MA, Dét. Synchrone, Analyse fréquentielle)

D'après les textes de THIEBAUT Bernard

INTRODUCTION GENERALE

Ce document regroupe l'ensemble des textes de Travaux Pratiques permettant de mettre en oeuvre et d'utiliser un processeur de traitement numérique du signal ou **Digital Signal Processor (DSP)**

Il y a 5 séances de Tp de 4 h qui vont successivement parcourir les thèmes suivants :

- *Prise en main du matériel et des outils de développement*
- *Écriture des premiers programmes*
- *Génération de signaux*
- *Implantation de filtres*
- *Autres exemples de TNS (8 heures)*

Il va s'agir ici de faire du traitement numérique en **temps réel** (et pas seulement en simulation) pour mieux appréhender les principaux problèmes (hardware et software) relatifs à une chaîne complète d'acquisition, de traitement et de restitution du signal.

Le travail s'effectuera a priori en binôme (à voir si besoin avec l'enseignant).

Les textes sont de difficulté progressive et il n'y a pas forcément prévu 1 texte par séance.

Évaluation

Chaque binôme devra faire un rapport sur son travail qu'il faudra remettre à l'enseignant à la fin de la série de TP.

La rédaction fera apparaître chaque fois les étapes de l'analyse des problèmes posés en utilisant les outils descriptifs appropriés : schémas, justifications théoriques, commentaires, analyse critique ...

Les résultats expérimentaux seront mis en valeur et analysés

Dès que celle-ci s'imposera, une conclusion partielle fera la synthèse des différentes étapes du travail demandé.

Documentation et exemples de codes

Normalement, la documentation complète (hardware et software) est présente sur chaque PC des TP dans le SHARC DOCUMENT LIBRARY et donc consultable à tout moment sur votre machine.

Important

Cette série de Tp va se dérouler à partir d'une machine virtuelle permettant de lancer la version adéquate de Windows et comprenant à la fois tous les utilitaires nécessaires, et aussi les documentations constructeurs.

Le seul texte externe est ce texte de Tp que vous retrouverez sur le bureau Windows.

Une fois cette machine virtuelle lancée, vous trouverez un dossier d_dsp qui regroupe des canevas ou des exemples de programmes (*.C, *.BAT et *.M) utiles pour ce TP.

Attention à bien sauvegarder l'ensemble de votre travail à la fin de chaque séance, idéalement sur clef USB et à venir avec à la séance suivante.

L'appel des différents outils (compilateur C avec les différentes options de compilation...) se fera en ouvrant une fenêtre DOS par des fichiers batch qui sont présents dans le dossier d_dsp .

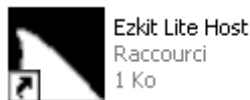
Pensez à utiliser des noms de programmes C évocateurs de la tâche à réaliser et surtout des noms ayant moins de 8 caractères sans espace (le compilateur indique une erreur sinon ...)

I. Prise en main des outils logiciels.

Afin de se familiariser avec l'environnement logiciel disponible, des petits exercices sont demandés. Dans cette partie, aucun code ne sera à écrire, et vous vous contenterez d'utiliser des logiciels de démonstration fournis par **Analog Devices**.

Le PC étant hors tension, relier tout d'abord la carte cible au PC puis alimenter cette carte et allumer votre PC.

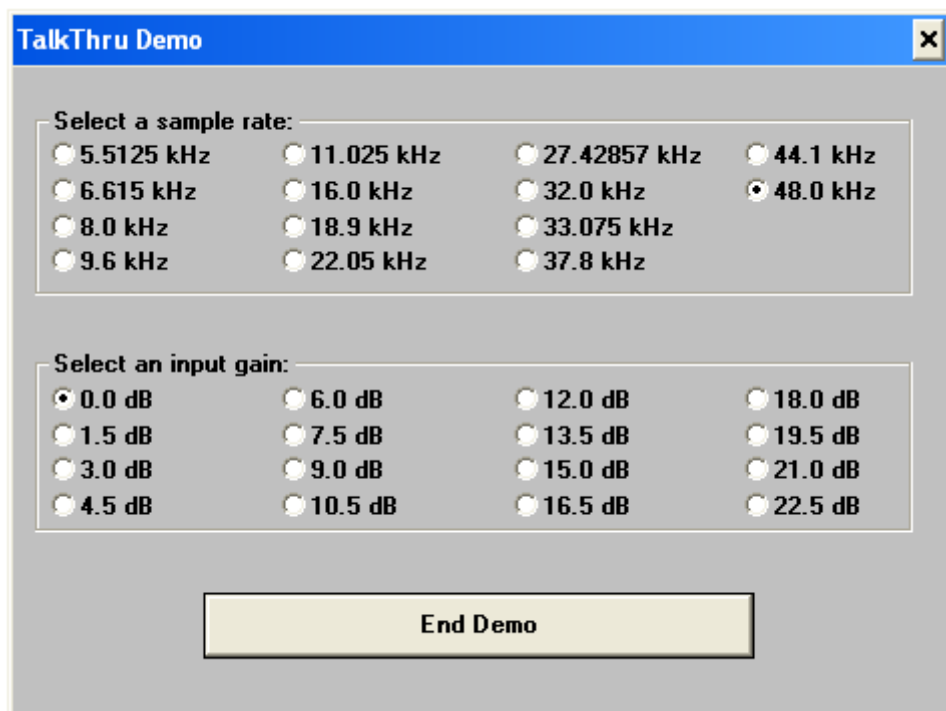
Lancer le logiciel de communication avec la carte cible DSP en cliquant sur l'icône qui se trouve dans le dossier **DSP** du bureau de Windows:



I.1. Test du logiciel Talkthru.

Faire ensuite **File / Demos / Talkthru** (ou taper directement sur l'icône ). Le logiciel **Talktru** est **téléchargé** automatiquement puis s'**exécute** sur la carte cible.

L'écran suivant apparaît alors sur votre PC :



Il permet de sélectionner une fréquence d'échantillonnage (**Select a sample rate**) et/ou un gain (**Select an input gain**) parmi quelques valeurs proposées.

On se rappellera (cf. schéma de principe de la page suivante et à rapprocher aussi de la figure V.11. du polycopié de cours) que :

- rx_buf[1] contient les échantillons $x_L(nT_e) = x_L(n)$ prélevés et convertis par le **CAN** sur le signal analogique $x_L(t)$ injecté sur la voie **gauche** du connecteur « jack » d'entrée **J23**,

- tx_buf[1] contient les échantillons $y_L(nT_e) = y_L(n)$ envoyés vers le **CNA** pour fabriquer le signal analogique $y_L(t)$ présent sur la voie **gauche** du connecteur « jack » de sortie **J22**.

Pour rx_buf[2] et tx_buf[2], c'est le même principe mais pour la voie **droite**.

Cette démo **Talkthru** recopie en fait les signaux présents sur les entrées gauche et droite respectivement sur les sorties gauche et droite. En fait, le logiciel **Talkthru** effectue tout simplement :

$$\text{tx_buf}[1] = \text{rx_buf}[1] \quad \text{et} \quad \text{tx_buf}[2] = \text{rx_buf}[2]$$

(donc **aucun** traitement numérique proprement dit, ou plus précisément cela correspond à un filtre numérique dont l'équation aux récurrences serait seulement $y(n) = x(n) \forall n$).

Avant d'injecter tout signal sur la carte cible et pour ne pas endommager les étages d'entrée droite ou gauche de cette carte, régler un GBF en sinusoïdal de manière à

ne pas dépasser 2V crête-à-crête
(vérifiez d'abord à l'oscilloscope)

Par exemple, un signal sinusoïdal de 500V d'amplitude est une valeur nominale utilisable ici.

1. Première manip

En sélectionnant un gain de 0 dB (**Input Gain**) et un signal d'entrée dont la fréquence est compatible avec la fréquence d'échantillonnage que vous aurez sélectionnée (**Sample Rate**) sur **Talkthru**, visualiser **sur l'oscilloscope** les signaux d'entrée (GBF et voie **INPUT J23**) et de sortie (voie **OUTPUT J22**) et utiliser cette démo pour repérer correctement les entrées analogiques droite et gauche ainsi que les sorties analogiques droite et gauche de la carte cible DSP.

(Ne pas se fier aux couleurs des bornes sur les blocs plexi (qui peuvent varier d'un bloc à l'autre) : seule la masse, toujours reliée à la tresse du câble deux conducteurs qui est mis à votre disposition, est facilement repérable ...)

Dans ces conditions, quel devrait être le rapport théorique des amplitudes entre le signal de sortie et celui d'entrée ?

Mesurer **très précisément** le rapport expérimental obtenu, car cette valeur vous sera utile dans toute la suite de ce TP.

Expliquer ce résultat expérimental, différent de la théorie en vous aidant notamment du schéma de la carte DSP fourni par la documentation constructeur (SHARC **EZKIT REFERENCE MANUAL** puis **SCHEMATICS** (sur la page [160/172](#) repérer les entrées droite et gauche : entre le Jack Stéréo **J23** de l'**INPUT** et les entrées **RLINE** et **LLINE** du Codec).

Quel est le rôle du filtre passif de type RC (par exemple R18, R19 et C50 pour la voie gauche) à l'entrée de chacune des voies du CAN du Codec (représenté par le **Bloc de Mise En Forme IN (BMEF IN LEFT ou RIGHT)** dans le schéma ci-dessous) ?

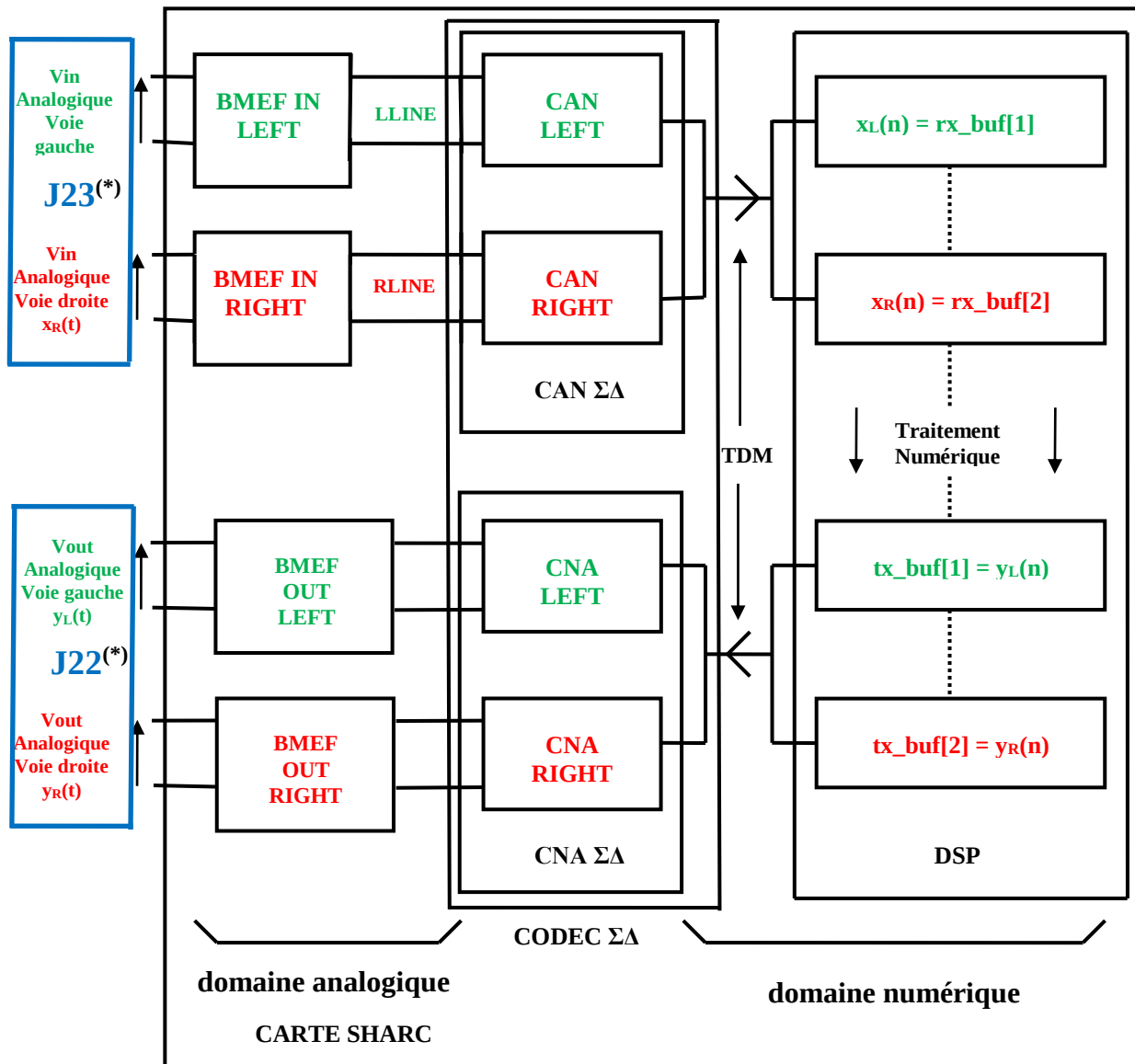


Schéma de principe du système SHARC de Analog Devices

Nb :

J22 et **J23** sont des connecteurs de type « Jack » stéréo à 3 points :

- un pour la voie **droite**,
- un pour la voie **gauche**,
- un pour leur **masse commune**.

2. Deuxième manip

Essayer divers gains, tout en vérifiant bien sûr toujours les critères de Shannon pour la fréquence d'échantillonnage. Choisissez des valeurs « simples » qui vous permettront de vérifier les résultats expérimentaux.

Notez que si on prend un gain trop grand, on risque de saturer le CAN du Codec et donc à éviter !!!

En vous aidant de la documentation du Codec **AD1847** présente sur votre PC, quel est le (ou les) registres du Codec qui permet(tent) de modifier ce gain ?

3. Essayer diverses fréquences d'échantillonnage pour une fréquence d'entrée fixe, par exemple 2 kHz. Regarder évoluer l'amplitude du signal de sortie par rapport au signal d'entrée. Conclusions.

Toujours en utilisant la documentation du Codec **AD1847**, quel est le registre du Codec qui permet de modifier la fréquence d'échantillonnage ? Et quels sont les bits de ce registre qui permettent ce choix ?

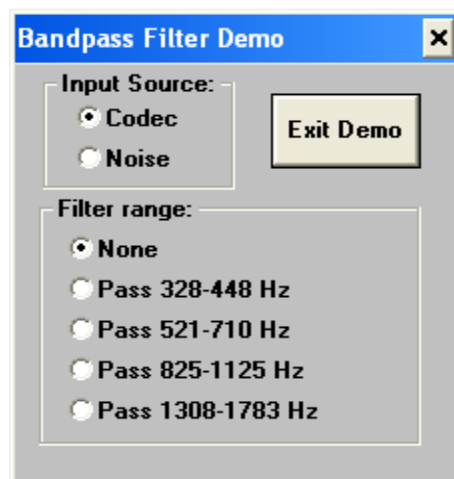
4. Pour une fréquence d'échantillonnage donnée, faire varier la fréquence du signal d'entrée. Conclusions.

I.2. Test du logiciel Talkthru.

1. Première manip

Faire maintenant **File / Demos / Bandpass Filter** (ou taper directement sur l'icône ). Le logiciel **Bandpass Filter** est téléchargé puis s'exécute sur la carte cible.

L'écran suivant apparaît alors sur votre PC :



Attention :

Cette démo de passe-bande ne marche en entrée que sur une des 2 voies, en l'occurrence la gauche.

À vous de déterminer la bonne voie, par exemple en utilisant None (pas de filtre) et un signal d'entrée à 1kHz car le Codec échantillonne à 8 kHz sur cette application.

Cette démo permet de télécharger un filtre à **réponse impulsionnelle finie** (RIF) sur le DSP.
 Vous avez le choix entre 4 types de filtre passe-bande, chacun de bande-passante entre $f_{\min}$ et $f_{\max}$.

L'objectif de cette partie est de comparer les performances de ce type de filtre **numérique** avec celles dont vous avez l'habitude avec les filtres **analogiques** passe-bande du second ordre que vous avez certainement déjà rencontré dans d'autres TP antérieurs.

Tracer rapidement sur le papier semi-logarithmique fourni le diagramme de Bode en module exprimé en dB du premier filtre proposé (phase non demandée) :

soit **328 Hz – 448 Hz centré** sur la fréquence $f_0 = \sqrt{f_{\min} \cdot f_{\max}} = \sqrt{328 \cdot 448} = 383 \text{ Hz}$.

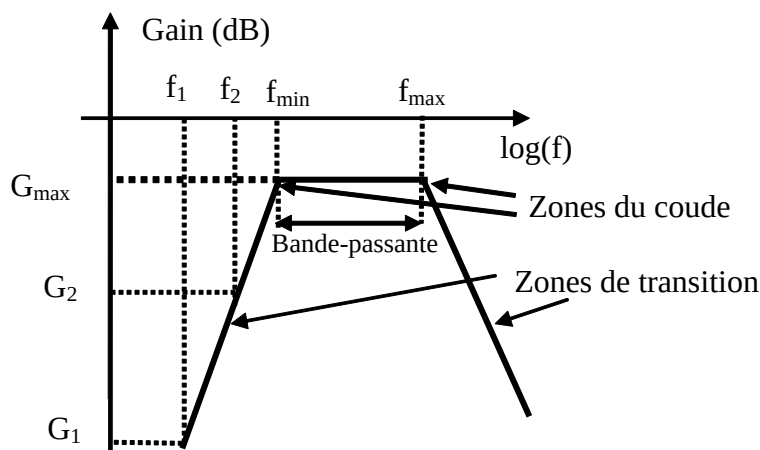
Ce tracé est à faire pendant la séance pour bien se rendre compte de la sélectivité de ce filtre numérique comparée à celle des filtres analogiques dont vous avez l'habitude.

On pourra se contenter des 7 points de mesure suivants, en prenant à chaque fois un signal d'entrée V_e d'amplitude 1 V par exemple, pour aller plus vite dans le calcul du gain $G_{dB} = 20 \cdot \log(V_s/V_e) \dots$

Fréquence (Hz)	280	300	$f_{\min}=328$	$f_0=383$	$f_{\max}=448$	500	520
Amplitude de V_s (V)							
Gain (dB)							

On fera les mesures les plus précises possibles dans la zone de transition du filtre (partie montante ou partie descendante du diagramme comme sur la figure ci-dessous) et **non dans la bande passante ou dans les zones de « coude » du diagramme car sinon on risque de faire des « fausses » mesures et de sous-estimer l'ordre de ce filtre.**

En pratique, cela revient à prendre les deux fréquences f_1 et f_2 les plus éloignées possibles des zones de « coude » de la courbe.



On admettra que la pente p (en dB/décade) de la zone de transition est donnée par :

$$p(\text{dB} / \text{décade}) = \frac{G_2(\text{dB}) - G_1(\text{dB})}{\log\left(\frac{f_2}{f_1}\right)}$$

Estimer la pente de la zone de transition grâce aux mesures précédentes et en déduire l'ordre du filtre réalisé.

Conclusions sur l'efficacité de ce filtre numérique.

2. Deuxième manip

En utilisant toujours cette démo **Bandpass Filter**, on va essayer de tracer le spectre d'un signal périodique **carré** en entrée en mesurant la décroissance des harmoniques en « 1/n ».

On pourra utiliser un des filtres fournis et **choisir correctement** la **fréquence fondamentale** du signal **carré** d'entrée pour visualiser ses harmoniques 2, 3, 4 et 5. (voir la méthodologie ci-dessous pour ces manipulations).

Méthodologie :

- calculer la fréquence centrale f_0 de votre filtre: $f_0 = \sqrt{f_{\min} \cdot f_{\max}}$ (moyenne géométrique)

- utiliser en entrée un signal carré de fréquence f_0 .

Que représente alors le signal de sortie du filtre ? Mesurer son amplitude.

- utiliser en entrée un signal carré de fréquence $\frac{f_0}{2}$.

Que représente alors le signal de sortie du filtre ? Mesurer son amplitude.

.- Poursuivre avec en entrée des signaux carré de fréquence $\frac{f_0}{2}$. Mesurer à chaque fois l'amplitude.

En déduire le spectre du signal mesuré du carré que l'on représentera .

Conclusions sur cette série de mesures.

II. Premiers programmes à partir de `canevas.c`

Cette partie va permettre de vous familiariser avec les entrées/sorties analogiques de l'EZ-KIT, de bien comprendre le mécanisme des interruptions matérielles (rappelez vous qu'une interruption sous DMA est générée à **chaque instant d'échantillonnage** c'est-à-dire pour tous les instants multiples de T_e et aussi qu'une deuxième interruption peut être générée par le **timer** interne du DSP).

Vous allez aussi faire vos premiers essais de traitement du signal.

Pour cela, on va reprendre ici quelques une des manipulations faites précédemment dans la **partie I.** de ce TP, mais réalisées maintenant sous votre propre contrôle et par votre propre programme.

Dans la machine virtuelle et dans le dossier adéquat (`prog_c`), vous utiliserez le batch **COMPIL** (qu'il ne faut pas changer) en **modifiant** le fichier `canevas.c` (cf. photocopié de cours complet pages 82 à 86) fourni comme base.

Vous préciserez bien les modifications apportées sur ce source (en ne reportant par exemple sur votre compte-rendu que les parties modifiées), et en **n'oubliant pas** d'utiliser des noms de programmes **évocateurs** comportant moins de 8 caractères, pour réaliser les exercices demandés dans la suite.

Ces programmes compilés (avec l'extension **.21K**) seront ensuite testés sur le DSP, après y avoir été téléchargés via le **EZKIT LITE HOST** par : **File, Open**.

On rappelle également que dans la version de base de `canevas.c`, le Codec échantillonne sur les deux voies d'entrée à **8 kHz**.

1. Recopier l'entrée gauche (**LIN**) sur la sortie gauche (**LOUT**), et vérifications.

Effectuez une mise à l'échelle par logiciel en utilisant le coefficient mesuré expérimentalement lors de la manipulation précédente avec le logiciel **Talkthru**, de façon à ce que les signaux d'entrée et de sortie aient la même amplitude.

2. Recopier l'entrée droite (**RIN**) sur la sortie droite (**ROUT**), et vérifiez après, bien sûr, avoir fait la mise à l'échelle.

3. Changer la fréquence d'échantillonnage.

Vous choisirez vous-même une nouvelle fréquence d'échantillonnage (voir pour cela le registre **Data Format** de la documentation du **Codec** et modifier les bits adéquats).

Notez bien que seules certaines fréquences sont proposées et donc implantables sur le DSP.

Vous indiquerez quels sont les bits à modifier, quel est leur nouvelle valeur, et à quel endroit du code cette modification doit être faite.

Rappel : par défaut, le programme **CANEVAS.C** effectue un échantillonnage à la fréquence de **8 kHz**.

Proposer une méthode pour vérifier expérimentalement la nouvelle fréquence d'échantillonnage que vous avez choisie.

Les parties 4 à 6 suivantes sont optionnelles (à ne faire que s'il y a le temps)

4. Lire la documentation de la fonction `timer`

`_set` dans la doc **ADSP_21000 FAMILY C RUNTIME LIBRARY MANUAL**.

(cette fonction permet d'initialiser les 2 registres **TPERIOD** et **TCOUNT** du timer interne du DSP).

Faire ensuite le lien avec le hardware du DSP en lisant aussi les informations sur le timer interne du DSP dans le **ADSP 2106X SHARC USER'S MANUAL** au sous-chapitre 3.7. du séquenceur de programme.

Aide : Vous essaieriez notamment de comprendre l'organigramme ci-dessous :

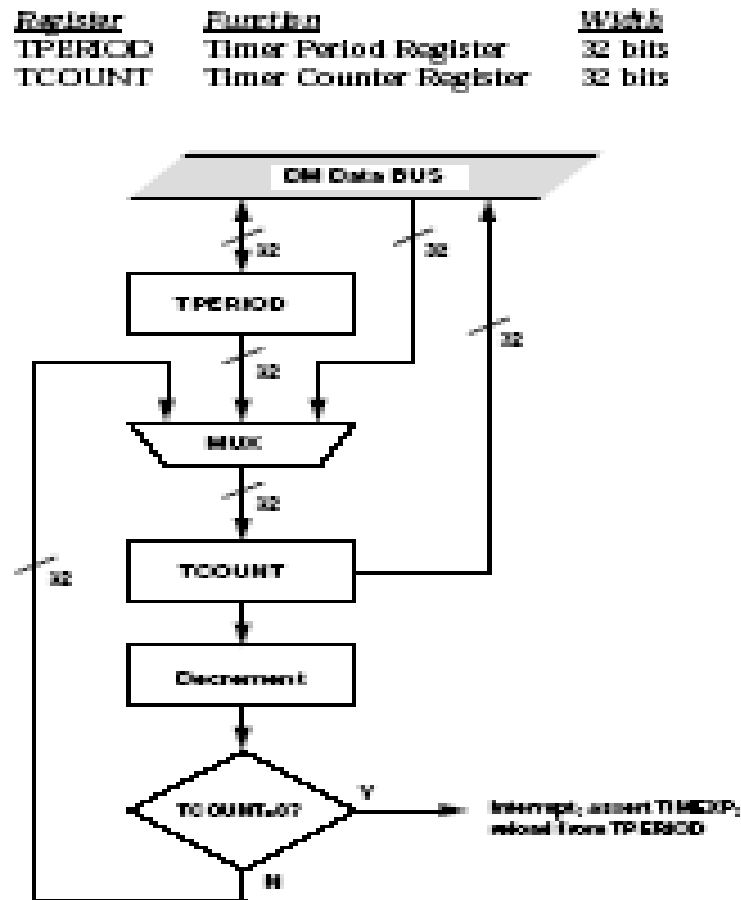


Figure 3.11 Timer Block Diagram

On essaiera de bien comprendre la différence entre les registres **TPERIOD** et **TCOUNT** et d'expliquer comment ces 2 registres fonctionnent. On se rappellera que sur la carte DSP, la fréquence du quartz est de **40 MHz**.

Mettez en application votre compréhension de la mécanique du timer en changeant la fréquence de clignotement de la LED **FLAG2** et en la faisant clignoter avec une période de **4** secondes (2 secondes allumée puis 2 secondes éteinte...) et vérifications.

Aide : voir à ce sujet la documentation sur la fonction **sets ADSP_21XXX flags** dans le **ADSP_21000 FAMILY C RUNTIME LIBRARY MANUAL** : le premier argument **SET_FLAGx** permet de sélectionner la LED concernée et le second l'action qui sera réalisée sur cette LED :

- **SET_FLAG** : pour l'éteindre,
- **CLR_FLAG** : pour l'allumer,
- **TGL_FLAG** : pour changer son état courant (si elle est allumée, on l'éteint et réciproquement),
- **TST_FLAG** : pour connaître, en retour, son état courant.

5. Faire clignoter alternativement les LEDS **FLAG1** et **FLAG2** en opposition de phase à la fréquence de votre choix **OU** réaliser un petit chenillard avec les LEDS **FLAG1**, **FLAG2** et **FLAG3** à la fréquence de votre choix.

6. Réalisez une « spirale » lumineuse **périodique** en faisant clignoter, **indéfiniment**, une des LED de plus en plus rapidement à partir de la période initiale précédente de 2 secondes (par exemple en suivant une progression géométrique de raison 0.9...).

III. Génération de signaux

III.1. Un exemple simple.

Toujours à partir de **CANVAS.C**, écrire un programme **GEN_SIN.C** qui génère de façon autonome (donc sans utiliser de GBF ni sur **LIN**, ni sur **RIN**) un signal **sinusoïdal** sur la sortie **LOUT** de **fréquence fixe** égale à **1 kHz** et d'**amplitude crête à crête** de **1V**.

L'expression temporelle en « temps continu » du signal de sortie sera donc :

$$y_L(t) = Y_m \cdot \sin(2\pi f_0 t) \text{ avec } Y_m = 0,5V \text{ et } f_0 = 1kHz.$$

Deux remarques et conseils:

- pour votre programme, le temps est discrétisé et vous n'avez accès, à chaque interruption DMA, qu'à des instants $t_n = n \cdot T_e$, tous multiples de la période d'échantillonnage T_e .

- penser à utiliser et à bien comprendre la **caractéristique de transfert** du CNA fournie à la page [55](#) du polycopié de cours complet (**et pas le résumé !!!**) pour le format 16 bits signés et linéaire utilisé ici...

Tester votre programme en mesurant la **fréquence** et l'**amplitude** du signal de sortie.. Analysez et commentez.

➔ On appellera l'enseignant pour lui montrer le résultat.

La partie suivante est optionnelle (à ne faire que s'il y a le temps)

Faire de même mais avec un signal **triangulaire symétrique**, à composante continue nulle et de même caractéristique sur sa sortie **ROUT** en écrivant un programme **GEN_TRI.C**.

Deux remarques et conseils:

- on pourra commencer à écrire l'expression par intervalle d'un signal triangulaire d'amplitude E et de période $T_0 = \frac{1}{f_0}$ en « temps continu » puis, comme dans le cas précédent, on « discrétisera » le temps,

- on aura ici avantage à échantillonner à $F_e = 32$ ou 48 kHz pour avoir plus d'échantillons par période du signal synthétisé et s'affranchir au mieux du filtre de mise en forme en sortie.

➔ On appellera l'enseignant pour lui montrer le résultat.

III.2. Synthèse avec deux interruptions : génération de sons.

On donne ci-dessous une table de notes de musique et de leurs fréquences (en Hz) pour 3 octaves :

Octave	3	4	5
DO	130,81	261,63	523,25
DO#	138,59	277,18	554,37
RE	146,83	293,66	587,33
RE#	155,56	311,13	622,25
MI	164,81	329,63	659,26
FA	174,61	349,23	698,46
FA#	185,00	369,99	739,99
SOL	196,00	392,00	783,99
SOL#	207,65	415,30	830,61
LA	220,00	440,00	880,00
LA#	233,08	466,16	923,33
SI	246,94	493,88	987,77

Ecrire un programme **MUSIC.C** qui génère un petit morceau de musique d'une dizaine de notes (choisies de préférence dans la même octave) avec une temporisation (durée de chaque note) dans un premier temps constante et égale à $\frac{1}{2}$ seconde par note, **pilotée par l'interruption du timer** (ce qui signifie que chaque note dure $\frac{1}{2}$ seconde).

Il s'agit donc ici de faire une synthèse entre :

- La génération d'un signal sinusoïdal (mais ici à des fréquences différentes),
- Les interruptions par le timer déjà programmé donc avec une durée de $\frac{1}{2}$ seconde (à chaque interruption, on changera la fréquence de la sinusoïde)

Pour indication, « Au clair de la lune » donne : « do – do – do – ré – mi – ré – do – mi – ré – ré – do – do – do – do – ré – mi – ré – do – mi – ré – ré – do – ré – ré – ré – ré – la – la – ré – do – si – la – sol – do – do – do – ré – mi – ré – do – mi – ré – ré – do. ». Mais vous pouvez trouver autre chose.

Visualiser et écouter cette « mélodie » sur **LOUT** et indiquer les problèmes rencontrés.

Dans la mesure du temps disponible :

Proposer une solution théorique pour associer une durée propre à chaque note (comme par exemple 1 seconde pour une « blanche », $\frac{1}{2}$ seconde pour une « noire » ...) puis valider et tester votre proposition théorique.

➔ On appellera l'enseignant pour lui montrer le résultat.

IV. Filtrage numérique.

Dans cette partie, nous nous proposons de réaliser puis d'implanter sur le DSP quelques algorithmes de filtrage numérique. (**RII** et **RIF**).

Nous allons adopter la démarche suivante :

- commencer par une **synthèse manuelle** : sur un exemple simple, lorsque l'ordre du filtre n'est pas trop élevé. L'intérêt est de bien comprendre la méthode de synthèse si on la réalise complètement pas à pas.

Cette première étape fournira **l'équation aux récurrences** du filtre numérique.

- ensuite (et ce sera plus souvent le cas en pratique), on utilisera **MATLAB** ou **Scilab** (soit pour vérifier le calcul manuel, soit pour déterminer directement les coefficients du filtre) qui nous permettra de **simuler** le filtre. En effet la ToolBox "Signal Processing" contient tous les outils nécessaires pour analyser et synthétiser les filtres analogiques et numériques, pour tester leurs performances (en amplitude et en phase) sur n'importe quelle séquence de signaux en entrée en terme de fonctions de transfert (Laplace ou transformée en Z), de pôles ou de zéros, ou en variables d'états.

- on plantera dans un premier temps les coefficients (calculés lors de la synthèse manuelle) sur le **DSP** en programmant là aussi « à la main » **l'équation aux récurrences** du filtre synthétisé et on vérifiera le fonctionnement et les performances de ce filtre **en temps réel** sur le **DSP**, notamment en les comparant avec les résultats de la **simulation** obtenus sous **MATLAB**,

- puis enfin, on utilisera les **fonctions spécifiques du C** du **DSP**, optimisées pour utiliser au mieux le hardware de ce composant et permettant d'implanter l'algorithme du filtre. Et on vérifiera là aussi, le fonctionnement du filtre en temps réel sur le **DSP**.

Cependant dès que la sélectivité, et donc l'ordre du filtre augmente, le nombre de ses coefficients étant alors plus important, il s'avèrera très utile **d'automatiser le couplage** entre **MATLAB** et le **DSP** en **évitant le « portage manuel »** des coefficients fournis par **MATLAB** vers votre programme en **C**.

Ce sera la dernière étape qui nous permettra de **réaliser très rapidement** la **synthèse** et **l'implantation** de n'importe quel type de filtre numérique sur le DSP.

Cette procédure **automatique** de **synthèse rapide** vous servira dans les exercices des parties **V.** et **suivantes** chaque fois qu'un filtre numérique devra être utilisé et implanté sur le DSP.

IV.1. Les différentes étapes sur un filtre RII basique.

IV.1.1. Synthèse manuelle.

Demander à votre enseignant une valeur de fréquence de coupure F_0 .

Puis calculer les coefficients d'un filtre passe-bas de fréquence de coupure $F_c = F_0$ à partir d'un filtre analogique de **Butterworth d'ordre 2**.

Ce calcul nécessite la connaissance de la fréquence d'échantillonnage F_e qu'il faudra donc connaître a priori. Discutez donc avec votre encadrant du choix de cette fréquence.

On rappelle que la forme **normalisée** $\underline{H}(p_n)$ de la fonction de transfert de ce filtre s'écrit :

$$\underline{H}(p_n) = \frac{1}{1 + \sqrt{2} p_n + p_n^2} \quad \text{avec } p_n = \frac{p}{2\pi F_c} \text{ « pulsation de Laplace normalisée ».}$$

Deux remarques importantes, avant de démarrer votre calcul :

a) Comme on l'a vu précédemment (cf. fenêtre de Talkthru page 5 de ce polycopié de TP), n'oubliez pas que seules certaines fréquences d'échantillonnage F_e sont disponibles sur le CAN.

b) Penser également à bien choisir votre fréquence d'échantillonnage F_e par rapport à la fréquence de coupure F_c du filtre. Pour pouvoir tester votre filtre sur une large plage de fréquences, vous avez intérêt à prendre le rapport $\frac{F_e}{F_c}$ le plus grand possible.

La méthode à utiliser est **au choix** :

- soit la **transformation bilinéaire**.

Le changement de variable $p = 2F_e \frac{z-1}{z+1}$ dans $\underline{H}(p_n)$ permet de trouver la fonction de transfert (en z) $\underline{H}(z)$ du filtre.

Puis d'établir l'équation aux récurrences recherchée.

- soit par **invariance impulsionnelle**,

Après avoir décomposé $\underline{H}(p_n)$ en éléments simples, on passe au domaine temporel par la transformée de Laplace inverse pour obtenir la réponse impulsionnelle $h(t)$ du filtre analogique de Butterworth.

Il suffit ensuite « d'échantillonner » cette réponse impulsionnelle, ce qui revient à remplacer les expressions de la forme e^{at} par leur transformée en z de la forme $\frac{1}{1 - e^{aT_e} z^{-1}}$.

Et donc d'obtenir la fonction de transfert (en z) $\underline{H}(z)$ du filtre.

Il faut ensuite penser à renormaliser cette fonction pour obtenir un gain statique (aux basses fréquences) de 1 ($\lim_{z \rightarrow 1} \underline{H}(z) = \lim_{p_n \rightarrow 0} \underline{H}(p_n) = 1$).

Puis on peut enfin établir l'équation aux récurrences recherchée.

Quelque soit la méthode utilisée, en déduire l'équation aux récurrences permettant de réaliser l'implantation de ce filtre numérique. On exprimera la sortie $y(n)$ du filtre à chaque instant $t_n = n.T_e$ en fonction des entrées $x(n)$, $x(n-1)$, et des sorties $y(n-1)$, (filtre RII).

IV.1.2. Vérification sur Matlab ou Scilab

Deux remarques importantes à lire avant de commencer cette partie :

Remarque 1 : quelques exemples de fichiers Matlab sont fournis dans le répertoire correspondant. Vous pouvez les consulter. Aucun ne correspond exactement à ce que l'on vous demande de faire. Il faudra donc les adapter, notamment le fichier **OR_BUTT.m** qui permet entre autre de dénormaliser l'échelle de fréquences. Utiliser pour cela la fonction **freqz** pour obtenir la réponse en fréquence d'un filtre dont on connaît l'équation aux récurrences (c'est-à-dire les coefficients « a et b »).

Remarque 2 : Attention, il faut se rappeler qu'en convention Matlab les coefficients du filtre correspondent à une fonction de transfert en z de la forme « $\frac{b}{a}$ » c'est-à-dire :

$$\underline{H}(z) = \frac{b(1) + b(2)z^{-1} + \dots + b(m)z^{-m+1}}{1 + a(2)z^{-1} + \dots + a(n)z^{-n+1}}$$

Dans votre polycopié de traitement numérique du signal, c'est peut-être la forme « $\frac{a}{b}$ »:

$$\underline{H}(z) = \frac{a(0) + a(1)z^{-1} + \dots + a(m)z^{-m}}{1 - b(1)z^{-1} - \dots - b(n)z^{-n}}$$

1. Implanter sur Matlab les coefficients que vous avez obtenus par calcul manuel dans la partie précédente et vérifier les performances par simulation. (fonction **freqz**).

2. **Relever** ensuite rapidement la réponse en fréquence sur du papier semilog et sur une échelle dénormalisée en fréquence (ce qui permettra de faciliter la comparaison avec les résultats expérimentaux obtenus par la suite sur le DSP).

On notera notamment la **fréquence de coupure à -3dB** du filtre et on comparera avec celle qui a servi pour effectuer le calcul manuel des coefficients.

Si les résultats sont satisfaisants, on pourra « sauter » la partie suivante et passer directement au **V.1.4.** Sinon, il faut faire calculer les coefficients du filtre par Matlab en passant à la partie suivante (**V.1.3.**).

➔ On appellera l'enseignant pour lui montrer le résultat.

IV.1.3. Utilisation de Matlab.

1. **Faites calculer** les coefficients de ce filtre par Matlab. On pourra utiliser la fonction **butter** très utile pour faire une synthèse de filtre numérique RII à partir d'un gabarit analogique.

2. **Vérifier** les performances par simulation.

3. On **relèvera** ensuite rapidement la réponse en fréquence sur du papier semilog et sur une échelle dénormalisée en fréquence (ce qui permettra de faciliter la comparaison avec les résultats expérimentaux obtenus par la suite sur le DSP).

On notera notamment la **fréquence de coupure à -3dB** du filtre et on comparera avec la valeur théorique souhaitée.

→ On appellera l'enseignant pour lui montrer le résultat.

IV.1.4. Implantation sur le DSP.

1. **Planter** votre équation aux récurrences sur un programme en C (soit celle obtenue manuellement et qui a été validée par Matlab, soit celle qui vous a été fournie par Matlab si votre calcul manuel n'était pas bon ...)

Remarque : n'oubliez pas que le gain statique du filtre doit valoir 1 (aux basses fréquences). C'est encore le moment d'utiliser le coefficient que vous avez mesuré précisément avec le logiciel **Talkthru** !!!

2. Tester les performances de votre filtre sur le DSP (tracé du diagramme de Bode expérimental **sur le même graphe** que le tracé précédent fourni par la simulation ...).

Comparer avec la simulation et commenter les résultats expérimentaux.

→ On appellera l'enseignant pour lui montrer le résultat.

IV.1.5. Utilisation de la fonction IIR.

Pour faciliter l'implantation des filtres numériques, pour éviter l'écriture « à la main » parfois fastidieuse de l'équation aux récurrences (comme on vient de le faire) et pour exploiter au mieux le hardware et l'architecture interne du DSP, **Analog Devices** fournit une fonction **IIR**, valable pour les filtres à réponse impulsionnelle infinie.

Sa documentation est fournie sur les pages **105** et **106** du polycopié de cours complet (**et pas le résumé**). Penser aussi à lire les deux remarques de la page suivante...

Elle présente l'avantage d'être générique et optimisée en ce qui concerne l'utilisation des mémoires de code (**PM**) et/ou de données (**DM**), du **MAC** et des **DAG** du DSP.

Deux remarques importantes :

a) Attention, l'exemple fourni dans la documentation de la fonction **IIR** ne s'utilisera pas tel quel.

Avant de le mettre en œuvre et de l'adapter à votre problème, essayer de bien le comprendre et notamment de voir le rôle des différents tableaux déclarés :

- **a_coefs**,
- **b_coefs**,
- le rôle de la déclaration **pm**,
- et **surtout** à quoi sert le tableau **state**.

b) pour le DSP (fonctions **IIR** et **FIR**), la forme pour la fonction de transfert en z est la suivante :

$$\underline{H}(z) = \frac{b(n) + b(n-1)z^{-1} + \dots + b(0)z^{-n}}{1 - a(n-1)z^{-1} - \dots - a(0)z^{-n}}$$

Noter bien les différences (le signe (dénominateur), la taille et l'ordre de rangement des coefficients) avec la convention de Matlab qu'on vous avait rappelée à la page 24 de ce polycopié de TP... Ceci est une source d'erreurs fréquentes.

Conséquence :

Faites bien attention en passant d'une convention à une autre!

Utiliser la fonction **IIR** (sur le filtre de Butterworth précédent) au sein d'un programme en C et tester ses performances.

→ On appellera l'enseignant pour lui montrer le résultat.

IV.1.6. Automatisation du couplage Matlab/DSP.

L'utilisation de la fonction **IIR** nécessite d'initialiser « manuellement » les tableaux de coefficients (**a_coefs**, **b_coefs**) en portant et adaptant ceux obtenus sous Matlab.

Si le nombre de coefficients est faible (comme ici jusqu'à maintenant car le filtre qu'on implantait était d'un ordre peu élevé), cela peut être fait sans risque d'erreurs.

Mais dès que l'ordre augmente (et on verra plus loin qu'on peut faire des filtres **RIF** d'ordre 256 ...), le nombre de coefficients à traiter augmente aussi et les sources d'erreurs avec. et ce n'est pas acceptable

On va donc demander à Matlab de créer (**fopen**) deux fichiers (que vous sauvegarderez de préférence dans votre sous-répertoire **PROG_C**, où se trouvent déjà tous vos programmes écrits en C) :

- un fichier **a_dsp.txt** : contenant la liste des coefficients « a » du filtre :
au format ASCII (utiliser **num2str**),
rangés dans le bon ordre (**fliplr**) (et en supprimant **a(1)**),
avec le bon signe
et séparés par une virgule,

- un fichier **b_dsp.txt** : contenant la liste des coefficients « b » du filtre :
au format ASCII (**num2str**),
rangés dans le bon ordre (**fliplr**)
et séparés par une virgule.

Pour cela, il faudra utiliser (comme indiqués ci-dessus) les fonctions de Matlab suivantes, dont on lira avant tout la documentation :

fopen, fwrite, fclose, fliplr, num2str... au sein d'un fichier *.m.

Avant d'utiliser les fichiers **a_dsp.txt** et **b_dsp.txt** (calculés en utilisant toujours le filtre de Butterworth du départ), il est conseillé de les éditer afin de vérifier s'ils ont le bon format ...

Ensuite, l'initialisation des tableaux a_coeffs (ou b_coeffs) se fera dans votre programme en C par une séquence du type (**syntaxe à bien respecter**) :

```
float pm a_coeffs[N] = {  
#include "E:\Le chemin vers votre_repertoire\d_dsp\prog_c\a_dsp.txt"  
};
```

Cette séquence remplace en fait l'initialisation dans le programme des tableaux de coefficients (liste manuelle des coefficients séparés par une virgule) par votre fichier créé sous Matlab.

Ce qui « automatisera » complètement le couplage entre Matlab et le DSP.

Vérifier votre chaîne de développement automatique en implantant le filtre de Butterworth d'ordre 2 et en vérifiant ses performances sur le DSP.

➔ On appellera l'enseignant pour lui montrer le résultat.

Partie suivante optionnelle : en fonction de l'avancement, on pourra directement passer à la partie suivante c'est-à-dire la synthèse de filtres RIF

IV.2. Application : conception et implantation d'un nouveau filtre.

1. Cahier des charges

→ À demander à votre enseignant

Synthétiser en approchant **au mieux*** le filtre-..... suivant :
bande : kHz- kHz.

Conseil : on pourra commencer avec un ordre 2 puis on essaiera de passer à un ordre plus élevé en augmentant le nombre de coefficients du filtre, **mais sans dépasser les capacités** du DSP.
(typiquement une douzaine de coefficients pour a et autant pour b).

2. On tracera enfin **rapidement** le diagramme de Bode du filtre ainsi réalisé.

→ On appellera l'enseignant pour lui montrer le résultat

IV.3. Filtrage RIF.

Quelques rappels sur la synthèse des filtres RIF.

On a vu que pour les **RII**, l'équation aux récurrences était de la forme :

$$y(n) = f(\underbrace{x(n), x(n-1), x(n-2), \dots}_{\text{entrées}}, \underbrace{y(n-1), y(n-2), \dots}_{\text{sorties}}).$$

Sous Matlab cette équation s'écrivait :

$$y(n) = b(1).x(n) + b(2).x(n-1) + b(3).x(n-2) + \dots - a(2).y(n-1) - a(3).y(n-2) - \dots$$

Les méthodes de synthèse de ces filtres **RII** étaient obtenues, comme on l'a fait avec le filtre de Butterworth, en utilisant (copiant) les gabarits de filtres analogiques connus.

Pour les filtres **RIF**, qui n'ont pas d'équivalents en analogique, les techniques de synthèse sont différentes.

Leur équation aux récurrences est de la forme :

$$y(n) = f(\underbrace{x(n), x(n-1), x(n-2), \dots}_{\text{entrées}}).$$

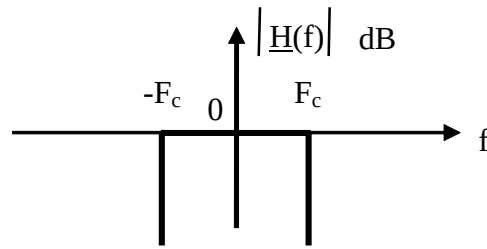
Seules les entrées (et leur historique) comptent, ce qui donne sous Matlab une équation de la forme :

$$y(n) = b(1).x(n) + b(2).x(n-1) + b(3).x(n-2) + \dots$$

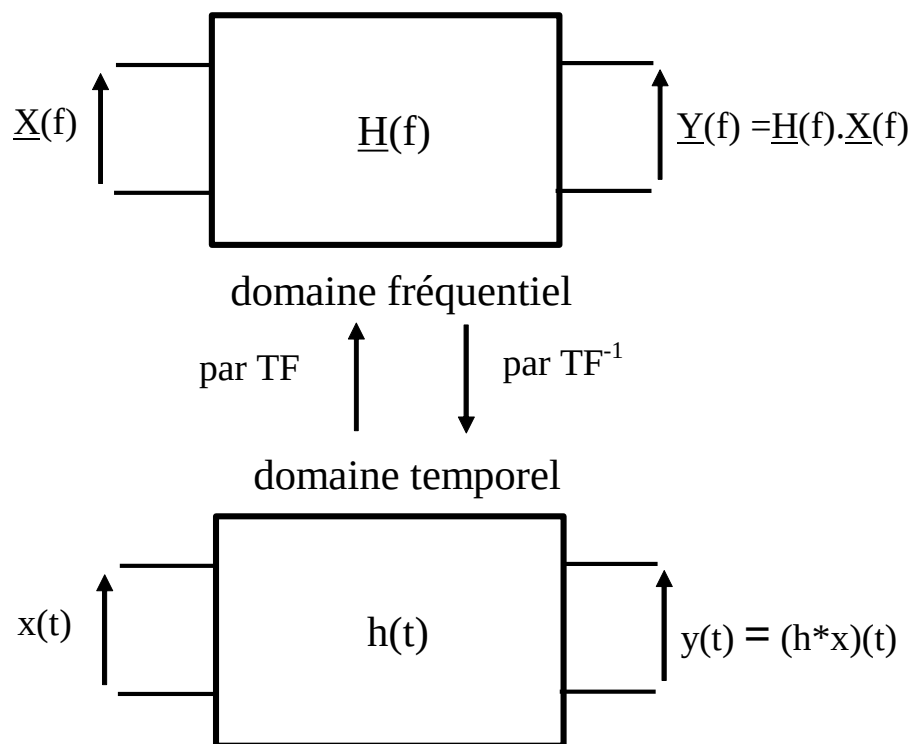
Les coefficients a(i) sont tous nuls (sauf a(1) = 1) : plus de dénominateur à la fonction de transfert en z !!!

Comme ce type de filtre n'existe pas en analogique, les méthodes de synthèse utilisent des gabarits de filtres « **idéaux** » et essaient de s'en approcher au mieux.

Par exemple, pour un passe bas de fréquence de coupure F_c , on utilisera le gabarit **idéal** suivant :



Ce qui donne formellement (en fréquentiel et en temporel) :



En fréquentiel, on aura :

$$\underline{Y}(f) = \underline{H}(f).\underline{X}(f) \quad (\text{avec les transformées de Fourier})$$

Ce qui donne, en temporel « analogique » :

$$y(t) = (h*x)(t) \quad (\text{produit de convolution analogique})$$

Soit :

$$y(t) = \int_{-\infty}^{+\infty} h(\tau).x(t - \tau)d\tau \quad (1)$$

$h(t)$ est la réponse impulsionnelle du filtre et peut se calculer en utilisant la Transformée de Fourier inverse TF^{-1} de $\underline{H}(f)$.

Sur ce même exemple, on obtiendrait :

$$h(t) = \int_{-\infty}^{+\infty} \underline{H}(f) \cdot e^{j2\pi ft} df = \int_{-F_c}^{+F_c} e^{j2\pi ft} df = \left[\frac{e^{j2\pi ft}}{j2\pi t} \right]_{-F_c}^{+F_c} = \left(\frac{e^{j2\pi F_c t} - e^{-j2\pi F_c t}}{j2\pi t} \right) = \frac{1}{\pi t} \cdot \sin(2\pi F_c t)$$

Et finalement :

$$h(t) = 2F_c \frac{\sin(2\pi F_c t)}{2\pi F_c t} = \text{fonction sinus cardinal sinc.}$$

On voit donc qu'en analogique, si $x(t)$ est connu, $y(t)$ est bien calculable par la formule (1).

Mais que se passe-t-il quand on passe en numérique ?

On transforme (1) en une somme « **discrète** » sur les instants d'échantillonnage $t_k = kT_e$ (produit **de convolution discret**)

$$y(n) = \sum_{k=-\infty}^{k=+\infty} h(k) \cdot x(n-k) \quad (2)$$

Malheureusement, cette équation, qui nous permet de calculer $y(n)$ pose 2 problèmes majeurs :

1. la sortie $y(n)$ du filtre est obtenue à partir d'une somme « **infinie** » de termes (ce qui est impossible à faire en un **temps fini**, même par le DSP le plus rapide ...)
2. et ... $y(n)$ nécessite (pour tous les $k < 0$) la connaissance de $x(n-k)$, c'est-à-dire des échantillons prélevés sur l'entrée à des instants **futurs** par rapport à $t_n = nT_e$, autrement dit le système à réaliser **n'est pas causal**.

1. Le **premier problème** est résolu en limitant k à une valeur **finie** N , ce qui revient à calculer $y(n)$ en utilisant $2N+1$ échantillons de l'entrée x .

On dit qu'on utilise une « **fenêtre** » d'observation **de largeur $2N+1$ échantillons** :

$$y(n) = \sum_{k=-N}^{k=+N} h(k) \cdot x(n-k) \quad (3)$$

(en général, on prendra $N = 2^p - 1$, comme dans la suite de ce TP pour le filtre passe-bas **RIF** à réaliser où $N = 15$ (31 échantillons)).

Ce qui donne en « cassant » cette somme en 3 parties :

$$y(n) = \underbrace{\sum_{k=-N}^{k=-1} h(k) \cdot x(n-k)}_{\text{futur}} + \underbrace{h(0) \cdot x(n)}_{\text{présent}} + \underbrace{\sum_{k=+1}^{k=+N} h(k) \cdot x(n-k)}_{\text{passé}} \quad (3)'$$

Pour avoir $h(k)$ ($= h(kT_e)$), il faut échantillonner en temporel la réponse impulsionnelle $h(t)$, ce qui revient à périodiser en fréquentiel la fonction de transfert $\underline{H}(f)$.

Dans ce cas, il faut utiliser les séries de Fourier (en effet $\underline{H}(f)$ est périodique, de période $F_e = \frac{1}{T_e}$). Et ainsi :

$$h(k) = \frac{1}{F_e} \int_{-F_e}^{+F_e} \underline{H}(f) \cdot e^{j2\pi f k T_e} df = 2 \frac{F_c}{F_e} \frac{\sin(2\pi k \frac{F_c}{F_e})}{2\pi k \frac{F_c}{F_e}} \text{ d'après le calcul précédent sur } h(t).$$

Soit :

$$h(k) = 2 \frac{F_c}{F_e} \text{sinc}(2\pi k \frac{F_c}{F_e}) \quad (4)$$

On peut donc, en numérique également, calculer par (4) les valeurs de $h(k)$.

À ce niveau, on peut enrichir la notion de « **fenêtrage** », que l'on peut symboliser dans le domaine temporel par une fonction $w(k)$ ($= w(kT_e)$) :

- si on utilise telles quelles les valeurs de $h(k)$ (obtenues par (4)) dans la formule (3), on dit que l'on prend une fenêtre rectangulaire (**boxcar**) c'est-à-dire que $w(k) = 1$ pour toutes les valeurs entières de $k \in [-N ; +N]$ et $w(k) = 0$ ailleurs. Ce qui revient à donner le même « **poids** » à tous les échantillons.

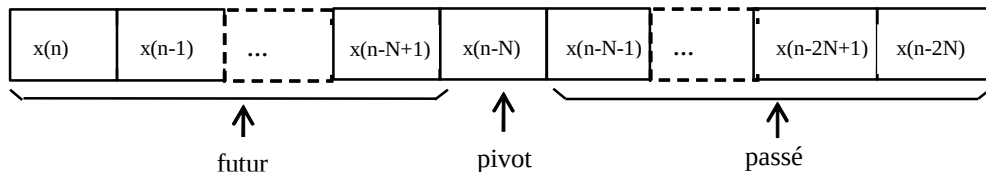
Mais à cause de cette discontinuité (pour $k = -N$ et $k = +N$) de la fenêtre dans le domaine temporel, on obtiendra des ondulations dans le domaine fréquentiel (phénomène de **Gibbs**) et donc la présence de lobes dits « secondaires » importants. Ce qui peut-être gênant, car un signal d'entrée de fréquence supérieure à F_c peut être interprété comme présent en sortie du filtre si ces lobes sont trop importants (ce problème est encore plus gênant dans les calculs de la FFT, comme on pourra le voir dans la dernière partie de ce TP : problème de la **résolution en amplitude**).

- mais d'autres types de fenêtres (triangulaire, Hamming, Blackmann ...) permettent de diminuer ces lobes secondaires (au prix d'une augmentation de la largeur du lobe central, ce qui peut-être gênant toujours dans le cas du calcul de la FFT : problème de **résolution en fréquence**). Toutes ces fenêtres ont la particularité de ne plus avoir cette discontinuité pour les valeurs de $k = \pm N$ et finalement de donner un « poids » moins important pour les échantillons les plus anciens.

En fait, comme souvent, le choix de la fenêtre résultera d'un compromis entre tous ces problèmes, y compris le nombre N qui caractérise la largeur temporelle de la fenêtre (en fait le nombre d'échantillons utilisés pour le calcul de la sortie $y(n)$ du filtre).

2. Le **deuxième problème** lié à la non causalité de l'expression (3)' est résolu en « bufférisant » (en gardant en mémoire l'historique des entrées) les échantillons $x(k)$ avant de faire le calcul de $y(n)$.

En effet supposons qu'on dispose à chaque instant $t_n = nT_e$ des $2N+1$ échantillons passés, comme indiqué ci-dessous :



(ce qui, on le voit sur ce schéma, revient à mémoriser $2N$ échantillons).

On considère que $x(n-N)$ correspond à l'instant « pseudo-présent » (le vrai présent étant bien sûr $x(n)$) et joue en quelque sorte le rôle de pivot car il présente l'avantage qu'à partir de cet échantillon on connaît à la fois :

- son **passé** (les $x(n-k)$ avec $N+1 \leq k \leq 2N$),
- mais **aussi son futur** (les $x(n-k)$ avec $0 \leq k \leq N-1$).

Finalement, cela revient à faire un décalage (retard) temporel constant $\tau = NT_e$.

Et c'est ce qui explique que les **FIR** déphasent le signal linéairement en fonction de la fréquence (déphasage $\varphi = 2\pi f \cdot \tau$).

L'équation (3)' devient alors :

$$y(n) = \sum_{k=-N}^{k=-1} h(k).x(n-N-k) + h(0).x(n-N) + \sum_{k=+1}^{k=+N} h(k).x(n-N-k)$$

Soit :

$$y(n) = \sum_{k=-N}^{k=+N} h(k).x(n-N-k) \quad (5)$$

Par ce retard temporel, le système est devenu **causal** et donc **réalisable**.

Or, l'équation aux récurrences de Matlab pour un **RIF** est :

$$y(n) = b(1).x(n) + b(2).x(n-1) + b(3).x(n-2) + \dots = \sum_{k=+1}^{k=2N+1} b(k).x(n-k+1) \quad (6)$$

Soit en ré-écrivant (5) en posant $j = k+N+1$: $y(n) = \sum_{j=+1}^{j=2N+1} h(j-N-1).x(n-j+1) \quad (7)$

En comparant (6) avec (7) il vient :

$$b(k) = h(k-N-1) \text{ pour } 1 \leq k \leq 2N+1$$

Et si on fait en plus un fenêtrage autre que rectangulaire, on aura :

$$b(k) = w(k-N-1).h(k-N-1) \text{ pour } 1 \leq k \leq 2N+1$$

Cahier des charges du filtre RIF à réaliser dans cette partie :

On se propose de réaliser un filtre **RIF** passe-bas :

- de **fréquence de coupure** $F_c = \frac{F_e}{4}$, avec $F_e = 8$ kHz pour avoir T_e assez grand et augmenter ainsi le temps de calcul disponible pour le DSP,
- et utilisant **$2N+1 = 31$ coefficients**.

IV.3.1. Simulation manuelle ou sous Matlab.

1. Calculer les coefficients $b(k)$ de ce filtre **RIF** :

- soit **manuellement** en utilisant les aspects théoriques vus plus haut,
- soit en utilisant la fonction **FIR1** de Matlab en précisant bien un fenêtrage de type **boxcar** (fenêtre rectangulaire) et en regardant avant tout la documentation de cette fonction dans l'aide de Matlab.

2. Puis vérifier son efficacité de filtrage **par simulation** en relevant **rapidement** sa réponse en fréquence sur du papier semilog et sur une échelle **dénormalisée** en fréquence (ce qui permettra de faciliter la comparaison avec les résultats expérimentaux obtenus par la suite sur le DSP).

On notera notamment la **fréquence de coupure à -3dB** du filtre et on comparera avec celle qui a servi pour effectuer le calcul manuel des coefficients.

3. Sauvegarder automatiquement les coefficients sous Matlab en utilisant une méthode analogue à celle vue précédemment pour les filtres RII (ici créer uniquement un fichier **b_dsp.txt** car les coefficients a sont nuls sauf $a(1) = 1$ bien sûr ...).

4. Tester sur Matlab les effets sur sa fonction de transfert de deux types de fenêtres : rectangulaire (**boxcar**) et Hamming (par défaut sur FIR1) en simulant sur le même graphe les courbes de réponse de votre filtre avec ces différentes fenêtres.

Commentez et comparez les résultats obtenus (sélectivité du filtre, importance des lobes secondaires ...)

➔ On appellera l'enseignant pour lui montrer le résultat !!!

IV.3.2. Implantation sur le DSP.

1. **Planter** ce filtre sur le DSP (fonction **FIR** dont la documentation est fournie à la page 104 du polycopié de cours) et vérifier ses performances en relevant sa fonction de transfert expérimentale sur la même courbe que celle de la simulation, mais d'une autre couleur. (on pourra utiliser un fenêtre de type **boxcar** dans un premier temps).

2. En utilisant la chaîne de couplage automatique précédente entre Matlab et votre programme en C, **améliorer** les performances de votre filtre en utilisant 255, voire 511 coefficients, **pour atteindre les limites de calcul** du DSP.

Conclusions.

→ On appellera l'enseignant pour lui montrer le résultat !!!

IV.3.3. Visualisation de la réponse impulsionnelle.

1. Que se passe-t-il si on envoie une **seule** impulsion de Dirac à l'entrée de ce filtre ?

2. Proposer une méthode pour visualiser la **réponse impulsionnelle** de ce filtre sur un oscilloscope.

Aide : il vous faut réfléchir à 3 conditions à respecter :

- amplitude maximum du signal d'entrée,
- durée des impulsions à envoyer en entrée,
- intervalle de temps entre chaque impulsion ...

On utilisera pour cela un générateur d'impulsions car les GBF utilisés classiquement en TP (ceux de chez HP en l'occurrence) ne permettent pas de réaliser des impulsions suffisamment fines... mais il faut impérativement régler le générateur d'impulsions correctement **avant de le relier** à la carte DSP pour éviter d'endommager l'étage d'entrée du CODEC.

Remarque : on peut aussi simuler le peigne de Dirac fourni par le générateur d'impulsions directement sur votre programme en C en utilisant judicieusement le buffer `rx_buf[]`. Comment?

3. **Comparer** le résultat obtenu sur votre oscilloscope avec la simulation de la réponse impulsionnelle de ce même filtre fournie par Matlab (fonction **filter**).

→ On appellera l'enseignant pour lui montrer le résultat !!!

V. Contrôle automatique de gain.

Tous les récepteurs (radio, TV, téléphone portable, récepteur IR ...) reçoivent un niveau de signal variable en fonction de différents paramètres comme l'éloignement de l'émetteur, l'atténuation plus ou moins importante due au canal de transmission, la présence éventuelle de multi-trajets dans le signal (réflexion, échos divers ...).

Si le niveau du signal reçu est faible, il faut que l'étage d'entrée ait un gain important pour fournir aux étages suivants un niveau suffisant. Par contre, si à un autre moment ou dans une autre configuration le niveau du signal d'entrée est plus important, le gain de l'étage d'entrée doit diminuer car sinon, en maintenant un gain important, on risque de saturer les étages suivants. Ayez en tête l'image de la « zapette » que vous pouvez utiliser à 10 m de distance pour changer de chaîne mais aussi à 1cm : on conçoit bien que le signal reçu à l'entrée du récepteur infrarouge au niveau du téléviseur n'aura pas, dans ces 2 cas extrêmes la même valeur et peut varier dans un rapport de 1 à 1000, voire plus.

Au final, on voit que le gain de l'étage d'entrée doit en quelque sorte s'adapter automatiquement à la puissance du signal reçu suivant une règle simple :

faible signal reçu grand gain,
fort signal reçu gain plus petit.

On parle de « **contrôle automatique de gain** » ou **CAG** en abrégé.

En pratique cette fonction de **CAG** est réalisée le plus souvent en analogique (on joue, grâce à sa polarisation sur la pente d'un transistor de type FET, et donc sur son gain).

Ici, on se propose de réaliser cette fonction en numérique grâce au DSP.

En utilisant un signal d'entrée $V_e(t)$ sinusoïdal d'amplitude et de fréquence variable, l'objectif sera d'obtenir en sortie un signal de même fréquence (système linéaire) mais d'amplitude constante (c'est en quelque sorte la **consigne**) en pilotant le gain du système à l'aide d'un algorithme de votre choix.

Mesurer la dynamique en amplitude et en fréquence du système réalisé en traçant dans le plan fréquence de V_e en abscisse, amplitude de V_e en ordonnée, la zone de fonctionnement de votre **CAG**.

Mesurer les limitations observées (en amplitudes et en fréquences), expliquer leurs causes, les problèmes rencontrés et les améliorations possibles pour obtenir un système plus performant.

➔ On appellera l'enseignant pour lui montrer le résultat !!!

VI. Modulation et démodulation d'amplitude

VI.1. Modulation par une simple multiplication.

Utiliser un signal sinusoïdal de fréquence $f_m=500$ Hz et d'amplitude 1V, fourni par un GBF, que vous injecterez en entrée de la carte DSP et que vous échantillonnerez par programme à $F_e=48$ kHz.

Changer le signe du signal échantillonné une fois sur 2 et visualiser en sortie le signal ainsi obtenu. Conclusions.

Que se passe t'il si la fréquence f_m augmente (par exemple jusqu'à 4 ou 5 kHz) ?

Aide : mesurer précisément la fréquence du signal de sortie. Expliquer le phénomène.

→ On appellera l'enseignant pour lui montrer le résultat !!!

VI.2. Modulation DSB-LC. (Dual Side Band – Large Carrier)

C'est la modulation d'amplitude "classique" avec double bande latérale et porteuse.

Nous allons réaliser cette partie en 3 étapes :

Étape 1 : A partir d'une porteuse :

$p(t) = A_c \cdot \cos(2\pi f_c t)$ créée par le **DSP**, réaliser une modulation DSB-LC d'un signal modulant $m(t) = A_m \cdot \cos(2\pi f_m t)$ fourni par un **GBF** en construisant et en visualisant le signal modulé $s(t)$ défini par :

$$s(t) = p(t) \cdot (1 + k \cdot m(t)).$$

On prendra : $f_c = 6$ kHz et $f_m = 600$ Hz.

On compilera comme d'habitude cette première version du programme avec le batch **COMPIL** en s'efforçant de prendre des valeurs pour k bien adaptées. (penser notamment aux problèmes du format des données envoyées vers le CNA de sortie et à leur normalisation). Inutile de passer à la suite si cette première version ne fonctionne pas correctement

Ensuite, et dans un deuxième temps, l'idée est de faire tourner sur le DSP un programme ayant besoin de certaines ressources du PC. Nous nous proposons de faire varier l'indice de modulation $m = k \cdot A_m$ en temps réel, c'est-à-dire à partir du clavier du PC par exemple.

C'est possible grâce au **DH21K** (cf. pages 77 à 78 du polycopié **de cours complet**).

Etape 2 :

Avant toute chose, prévoir une sortie « propre » du programme (par appui sur la touche Q ou ESC par exemple), car sinon, vous risquez de perdre la communication avec la carte DSP. Il suffit pour cela de rajouter dans le main de votre programme, à la place de la boucle infinie existante, une boucle du type :

```
do
{
c=getch();
.
.
.
}
while (c !=27); // code ASCII de la touche ESC
```

Etape 3 :

On va enfin écrire un programme **MODUL.C** qui utilise par exemple les touches ↑ et ↓ (ou toute autre combinaison de touches du clavier du PC) pour respectivement augmenter et diminuer m en temps réel.

A partir de maintenant, il faut utiliser le batch **DSPH.BAT** (qu'il faudra peut-être modifier en fonction de votre environnement de travail) pour compiler et télécharger votre logiciel sur l'EZ-KIT. (penser également à fermer le **SHARC EZKIT LITE HOST** car sinon il y aura conflit sur la liaison série reliant le PC à la carte DSP)

Remarquer qu'avec **DSPH**, le téléchargement se fait bien moins vite qu'avec le **SHARC EZKIT LITE HOST** car la liaison série ne fonctionne alors qu'à 9600 Bauds.

Ecrire le programme **MODUL.C** comme il est décrit plus haut.

➔ On appellera l'enseignant pour lui montrer le résultat !!!

VI.3. Détection synchrone.

Injecter un signal modulé en amplitude et fourni par un GBF, ayant les mêmes caractéristiques que dans la partie précédente ($f_c = 6 \text{ kHz}$ et $f_m = 600 \text{ Hz}$) sur une entrée de la carte DSP.

Effectuer une démodulation **synchrone** de ce signal.

Aide : pour la détection synchrone il vous faut :

- multiplier le signal d'entrée par une porteuse de même fréquence $f_c = 6 \text{ kHz}$ et **synchrone** avec ce signal,
- puis filtrer le résultat de cette multiplication par un passe-bas à concevoir.

Conclusions sur les performances de cette technique de démodulation.

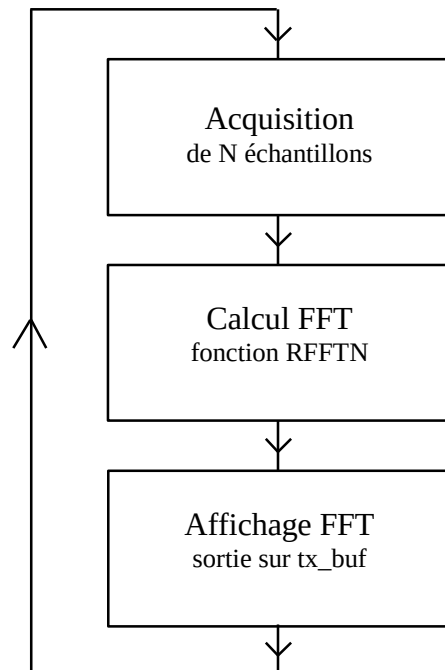
➔ On appellera l'enseignant pour lui montrer le résultat !!!

VII. Analyseur de spectres

Réaliser un **analyseur numérique** de spectres permettant de visualiser ces spectres sur un oscilloscope.

(fonction **RFFTN** avec $N = 2^p$ du C décrite page 107 du polycopié de cours).

Aide : on pourra concevoir un programme sous forme d'une boucle infinie dont la structure pourrait être de la forme suivante :



Vérifier les performances de votre analyseur de spectres avec un signal sinusoïdal, carré et triangulaire.

➔ On appellera l'enseignant pour lui montrer le résultat

Étalonner vos axes en amplitude et en fréquence.

Regarder l'influence du fenêtrage et du nombre d'échantillons prélevés sur les raies obtenues.
Conclusions.

➔ On appellera l'enseignant pour lui montrer le résultat

Fin des Tps de DSP